Lógica y Representación 1

Arreglos unidimensionales

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen





PROBLEMA

ALGORITMO



1. Entender el problema
2. Diseñar un plan
3. Implementar el plan
4. Revisar el plan

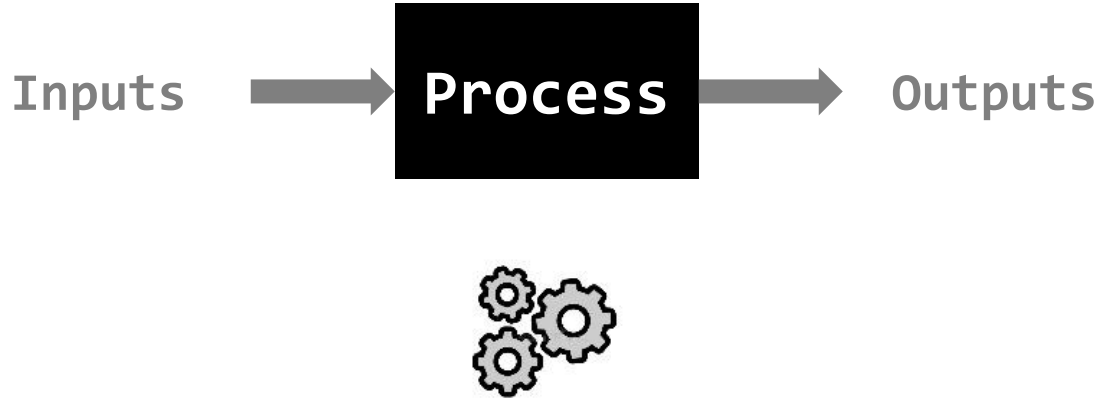
Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

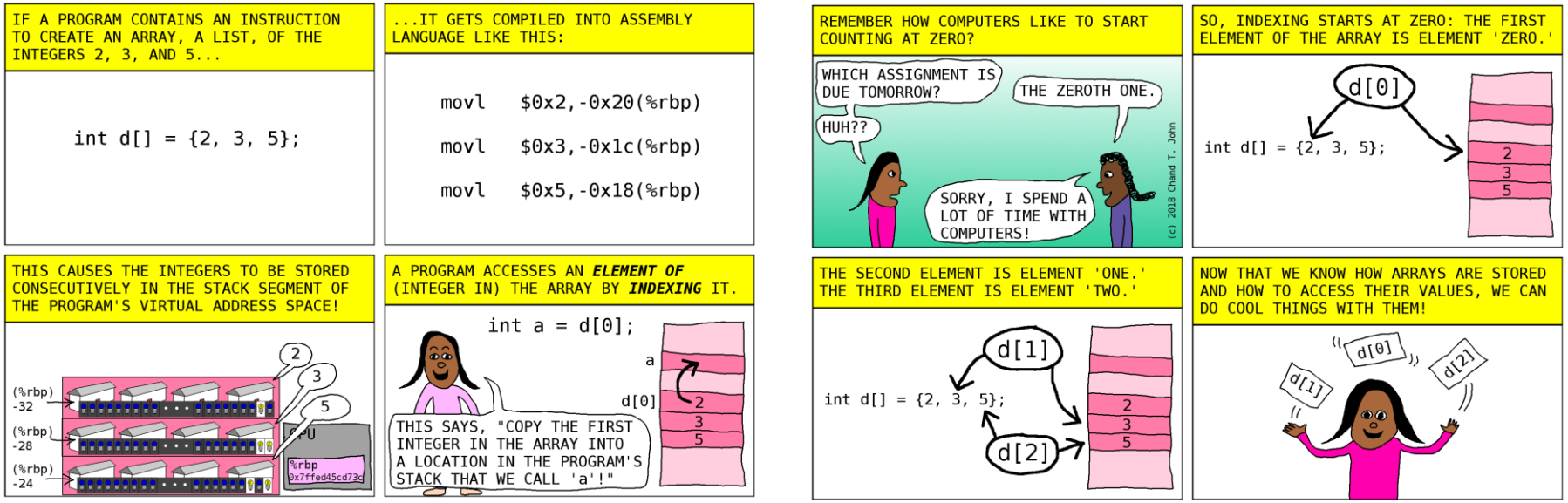
1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**







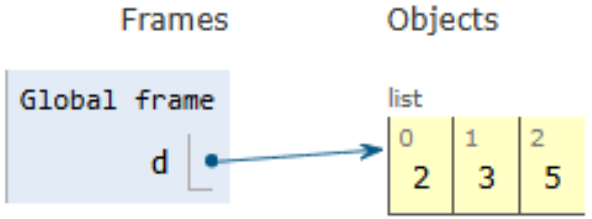
Arreglos unidimensionales




```
d = [2,3,5]
print(f"d[0] = {d[0]}")
print(f"d[1] = {d[1]}")
print(f"d[2] = {d[2]}")
```

Print output

```
d[0] = 2
d[1] = 3
d[2] = 5
```



[Link simulación]

Imagen: Chand John, **"How an Array of Integers Gets Stored in Memory"**, *CS Comics*, 2018. https://www.cs-comics.com/how-an-array-of-integers-gets-stored-in-memory/ — © CS Comics. Uso con fines educativos.

Lo que ya se sabe de funciones



ejemplo: `celsius_a_fahrenheit(100)` devuelve `212.0`

"Quien usa la función solo necesita su nombre, qué recibe y qué devuelve: no necesita ver su código."

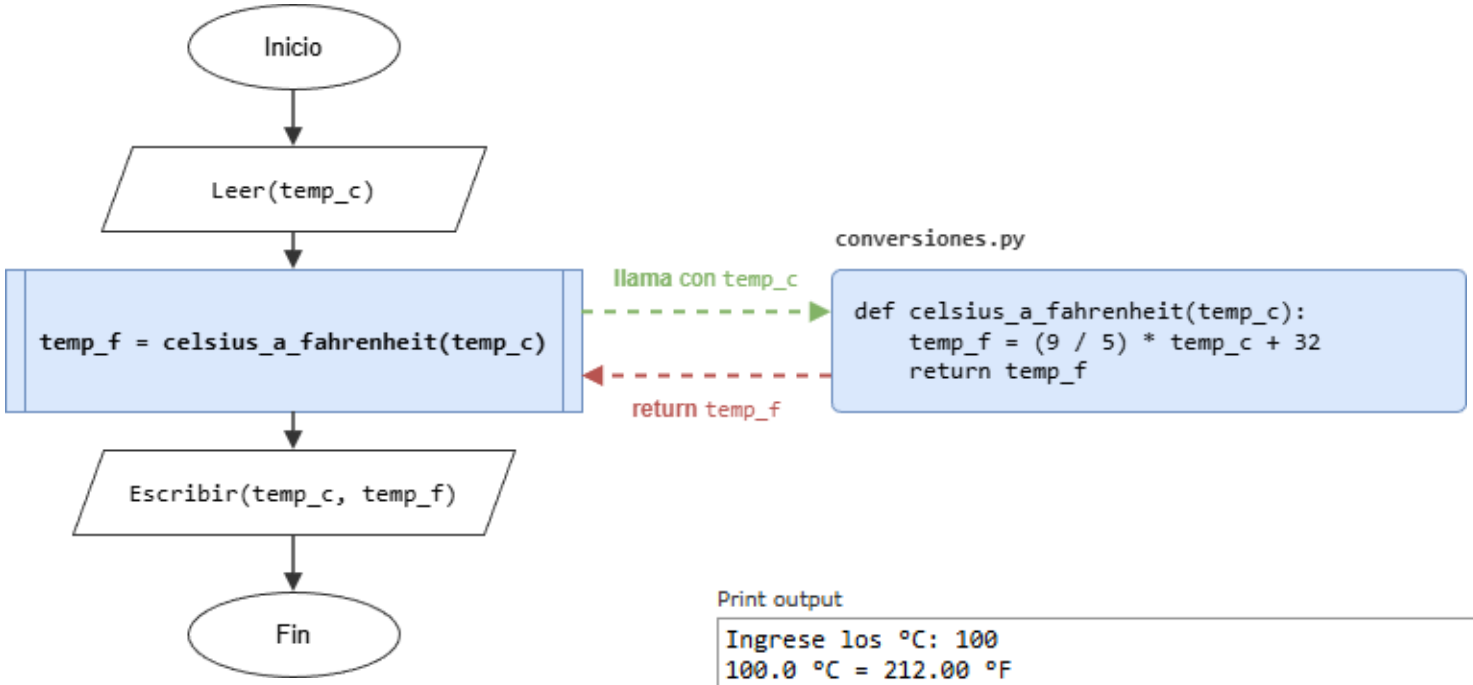
- Una función se define con **`def`** y entrega su resultado con **`return`**.
- Su nombre empieza con un verbo y dice qué hace.
- Su especificación (**`docstring`**) dice qué supone y qué garantiza.

conversiones.py

```
def celsius_a_fahrenheit(temp_c):
    """
    Convierte una temperatura de grados Celsius
    a grados Fahrenheit.

    Supone:
        temp_c es un numero (int o float) mayor
        o igual a -273.15 (el cero absoluto).
    Garantiza:
        Retorna la temperatura equivalente en
        grados Fahrenheit (float).
    """
    temp_f = (9 / 5) * temp_c + 32
    return temp_f
```

Quién llama y qué recibe de vuelta



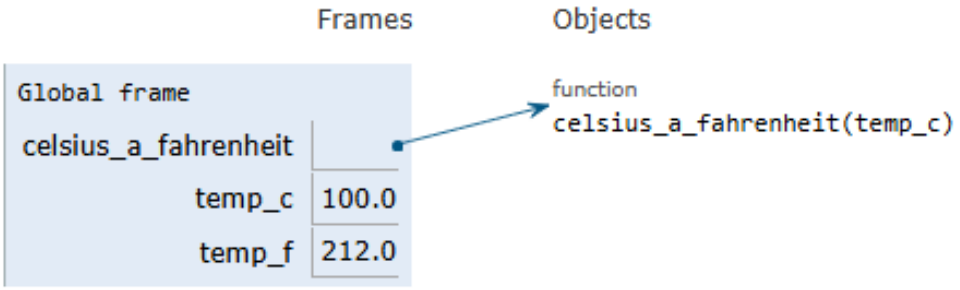

```
def celsius_a_fahrenheit(temp_c):
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)
print(f"{temp_c} °C = {temp_f:.2f} °F")
```

Link simulación

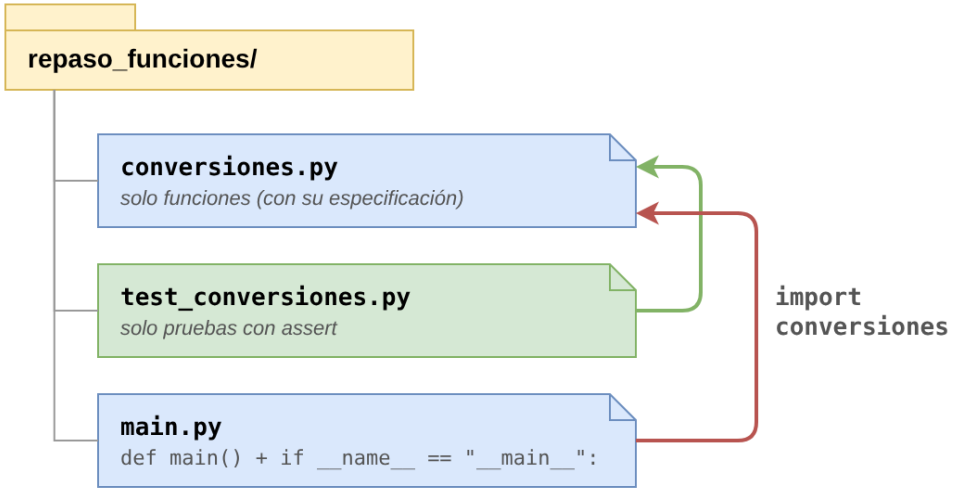
Print output

```
Ingrese los °C: 100
100.0 °C = 212.00 °F
```



Definir una función no la ejecuta: se ejecuta cuando se la llama. La llamada le entrega **`temp_c`** y recibe **`temp_f`** con **`return`**.

Un programa en varios archivos



*se ejecutan **solo las pruebas***

```
> python test_conversiones.py
Pruebas de conversiones: todas pasaron
```

*se ejecuta **solo el programa***

```
> python main.py
Ingrese los °C: 23
23.0 °C = 73.40 °F
```

test_conversiones.py

```
import conversiones

assert ...
...
print("Pruebas de conversiones: todas pasaron")
```

main.py

```
import conversiones

def main(): ...

if __name__ == "__main__":
    main()
```

Recordemos la convención de trabajo: De aquí en adelante, los programas que usen funciones se organizan en 3 archivos (**módulos**):

- **Funciones** (`conversiones.py`)
- **Pruebas con `assert`** (`test_conversiones.py`)
- **Programa principal** (`main.py`) con `def main()` y la guarda `if __name__ == "__main__":`

El problema que quedó pendiente

Enunciado: Leer las notas de los estudiantes de un curso y mostrar **cuántas notas son mayores que el promedio**.

- **Archivo**: `notas_sin_arreglos.py`

```
def main():
    cantidad = int(input("Cantidad de estudiantes: "))
    suma = 0.0
    for i in range(cantidad):
        nota = float(input(f"Nota del estudiante {i + 1}: "))
        suma += nota
    promedio = suma / cantidad
    print(f"Promedio del curso: {promedio:.2f}")
    # ¿Cuántas notas son mayores que el promedio?
    # Problema: la variable nota solo conserva la última nota leída.
    print(f"Nota guardada en la variable nota: {nota}")

if __name__ == "__main__":
    main()
```

- **Salida** (con 4 estudiantes: 3.0, 4.5, 2.0, 4.0):

```
Promedio del curso: 3.38
Nota guardada en la variable nota: 4.0
```

Pregunta: El promedio solo se conoce después de leer todas las notas. ¿Cómo se compara cada nota con el promedio, si las anteriores ya se perdieron?

Posible solución: Una variable por nota

Pregunta: El promedio solo se conoce después de leer todas las notas. ¿Cómo se compara cada nota con el promedio, si las anteriores ya se perdieron?

```
nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
nota3 = float(input("Nota 3: "))
nota4 = float(input("Nota 4: "))
promedio = (nota1 + nota2 + nota3 + nota4) / 4

cantidad = 0
if nota1 > promedio:
    cantidad = cantidad + 1
if nota2 > promedio:
    cantidad = cantidad + 1
if nota3 > promedio:
    cantidad = cantidad + 1
if nota4 > promedio:
    cantidad = cantidad + 1
print("Notas mayores que el promedio:", cantidad)
```





nota1 **3.0**

nota2 **4.5**





nota3 **2.0**

nota4 **4.0**

- Con 4 estudiantes funciona.
- Con 40 estudiantes serían 40 variables, 40 lecturas, una suma de 40 términos y 40 **`if`**. Aunque es posible, deja de ser practico.
- **No se puede usar un ciclo**: cada variable tiene un nombre distinto.

Observación: Si los datos se van a usar después de leerlos, hay que guardarlos todos.

Otro caso: Los empleados del señor Burns

Problema: El señor Burns quiere un programa que registre la identificación de cada empleado de la planta nuclear de Springfield.



`id_lenny` `"001"`



`id_carl` `"002"`



`id_homero` `"003"`



`id_barney` `"004"`



```
id_lenny = "001"
id_carl = "002"
id_homero = "003"
id_barney = "004"
# ... y una variable más por cada empleado nuevo
```

- Cada empleado nuevo obliga a crear una variable nueva y a cambiar el programa.
- ¿Y si la planta tiene 500 empleados?

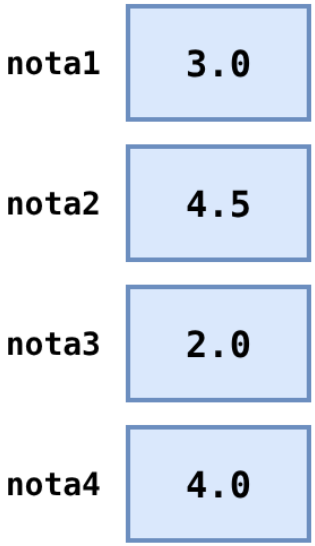
Pregunta: ¿Existe una forma de guardar muchos valores bajo un solo nombre?

¿Qué es un arreglo?

Un **arreglo** es una variable que guarda muchos valores bajo un solo nombre:

- Todos los valores son **del mismo tipo** (por ejemplo, todos reales).
- Se guardan en casillas **contiguas** (una al lado de la otra) en la memoria.
- Tiene un **tamaño fijo**: se decide al crearlo y no cambia.
- A cada valor se llega por su **posición** o **índice**.

Una variable por nota: **4** nombres

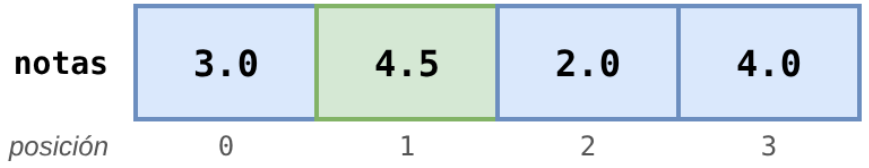


4 nombres distintos:
no se pueden recorrer con un ciclo

Un arreglo: un nombre, muchos valores

identificador: **`notas`**

tamaño: 4 *(posiciones de* `0` *a* 3*)*

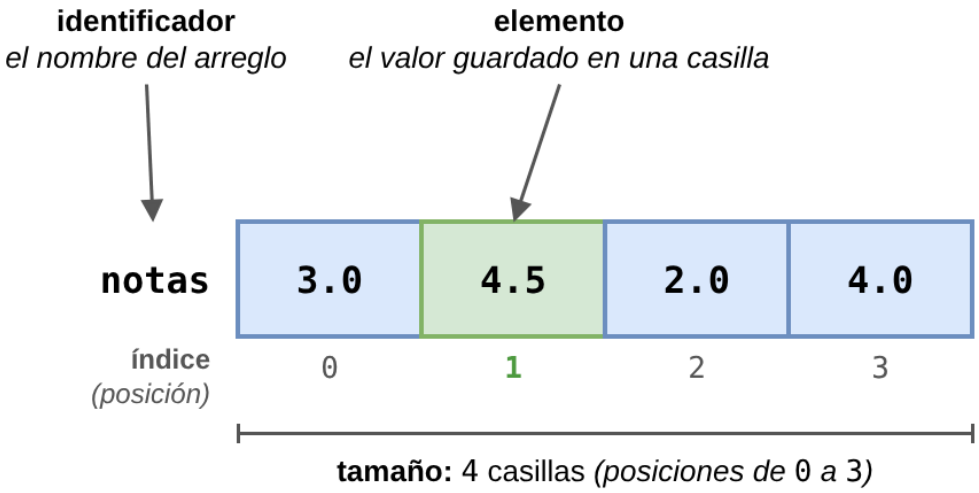


`notas[1]` → **4.5** *(se llega al elemento por su posición)*

un solo nombre: se recorre con un ciclo,
con la posición de 0 a 3

En Python: un arreglo se escribe con una lista: **`notas = [3.0, 4.5, 2.0, 4.0]`**

Partes de un arreglo



`notas[1]` → **4.5** *el elemento de la posición 1*

primera posición: `0` *última posición:* `N - 1 = 3`

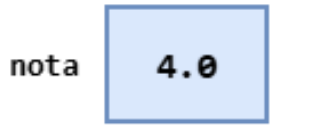
- **Vocabulario**:
  - **Identificador**: el nombre del arreglo, **`notas`**.
  - **Elemento**: cada valor guardado, por ejemplo **`4.5`**.
  - **Índice o posición**: el número de la casilla, en **`notas[1]`** el índice es **`1`**.
  - **Tamaño**: cuántas casillas tiene, **`4`**.

Observación: Las posiciones empiezan en **`0`**. Si el arreglo tiene **`N`** elementos, la última posición es **`N - 1`**.

Una variable frente a un arreglo

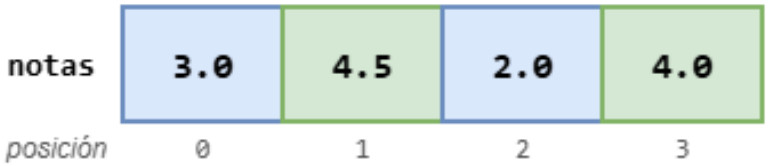
- **Una variable guarda un valor**: al leer una nota nueva, la anterior se pierde.
- **Un arreglo guarda todos los valores**: cada nota tiene su propia casilla.

Hasta hoy: una variable, **un** valor



las notas 3.0, 4.5 y 2.0 se perdieron

Desde hoy: una variable, **muchos** valores



promedio = 3.38 → **2 notas mayores** (posiciones 1 y 3)

- **Ventajas y desventajas de los arreglos**

Ventajas	**Limitaciones**
Muchos datos bajo un solo nombre	El tamaño es fijo: no crece ni se encoge
Se recorren con un ciclo, sin importar cuántos datos sean	Todos los elementos deben ser del mismo tipo
Se llega a cualquier elemento directamente por su posición	Un arreglo muy grande ocupa mucha memoria, aunque no se use completo

La lista usada como arreglo

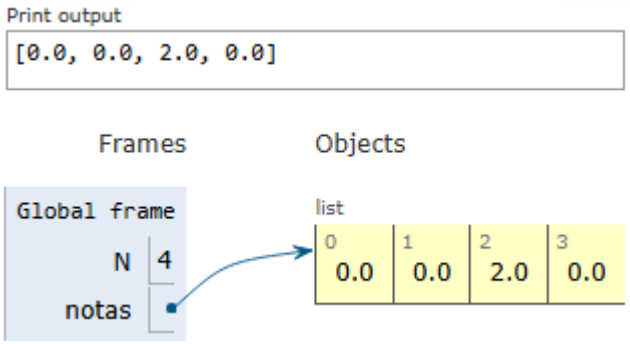
- Ya conocen las listas de Python. Una lista puede hacer **más cosas** que un arreglo. En este curso, por ahora, se usa **como arreglo**, con estas reglas:

La lista en Python permite…	**Usada como arreglo, en este curso…**
Crecer y encogerse: **`append()`**, **`remove()`**	**Tamaño fijo**: se crea con **`[0.0] * N`** y no cambia
Mezclar tipos: **`[1, "oso", 2.5, True]`**	**Un solo tipo**: todos reales, todos enteros, etc.
Índices negativos: **`notas[-1]`**	**Acceso por posición** de **`0`** a **`N - 1`**

- Las listas traen sus propias operaciones, como **`notas.append(5.0)`**. Son operaciones de objetos y se verán en **Programación Orientada a Objetos**.

```
N = 4
notas = [0.0] * N  # 4 casillas, todas en 0.0
notas[2] = 2.0     # se llega a la casilla por su posicion
print(notas)       # [0.0, 0.0, 2.0, 0.0]
```

Link simulación



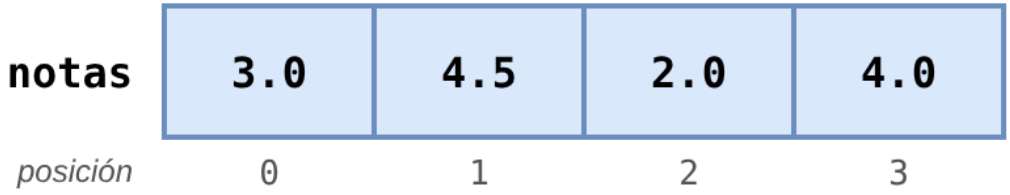
Idea clave: En Python el arreglo se escribe con una lista, pero la disciplina del arreglo la pone el programador.

Crear un arreglo

Qué se hace	Pseudocódigo	Python
Definir el tamaño	`N = 4`	`N = 4`
Crear el arreglo con N casillas	`notas = Arreglo(N)`	`notas = [0.0] * N`

Inicializar elemento por elemento	`notas[0] = 3.0`
	`notas[1] = 4.5`
	`notas[2] = 2.0`
	`notas[3] = 4.0`
		`notas[0] = 3.0`
		`notas[1] = 4.5`
		`notas[2] = 2.0`
		`notas[3] = 4.0`

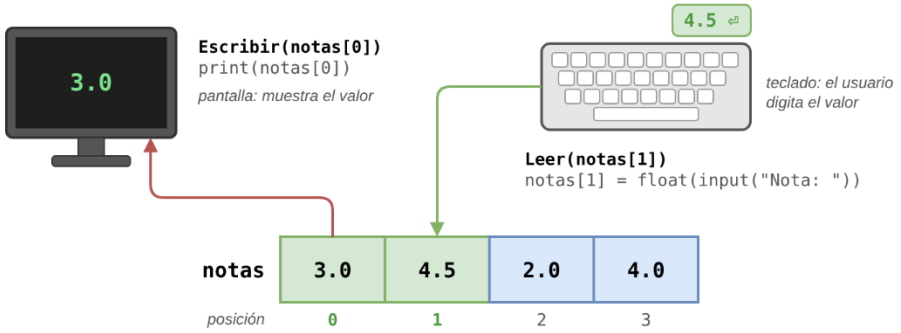
Crear e inicializar a la vez	`notas = [3.0, 4.5, 2.0, 4.0]`	`notas = [3.0, 4.5, 2.0, 4.0]`



- **`Arreglo(N)`** crea un arreglo con **`N`** casillas. El tamaño se fija al crearlo.
- En Python cada casilla necesita un valor inicial. Por eso **`Arreglo(N)`** se escribe **`[0.0] * N`**: el operador **`*`** repite el valor y forma una lista con **`N`** ceros.
- Si ya se conocen los valores, se crea e inicializa a la vez, escribiéndolos entre corchetes. El primer valor queda en la posición **`0`**.

Guardar y leer valores

Qué se hace	Pseudocódigo	Python
Leer desde el teclado y guardar en una casilla	`Leer(notas[1])`	`notas[1] = float(input("Nota: "))`
Mostrar el valor de una casilla	`Escribir(notas[0])`	`print(notas[0])`
Copiar el valor de una casilla a una variable	`primera = notas[0]`	`primera = notas[0]`
Usar las casillas en una expresión	`suma = notas[0] + notas[1]`	`suma = notas[0] + notas[1]`




```
N = 4
notas = [0.0] * N
notas[0] = 3.0
notas[1] = float(input("Nota: "))
print(notas[0])
primera = notas[0]
suma = notas[0] + notas[1]
print("Suma:", suma)
```

Link simulación

- **Cada casilla se usa como una variable normal**: se lee, se muestra, se copia y se usa en expresiones.
- Solo cambian **`Leer/Escribir`** por **`input()/print()`**. El acceso con corchetes **`notas[i]`** es igual en los dos.

El índice puede ser una expresión

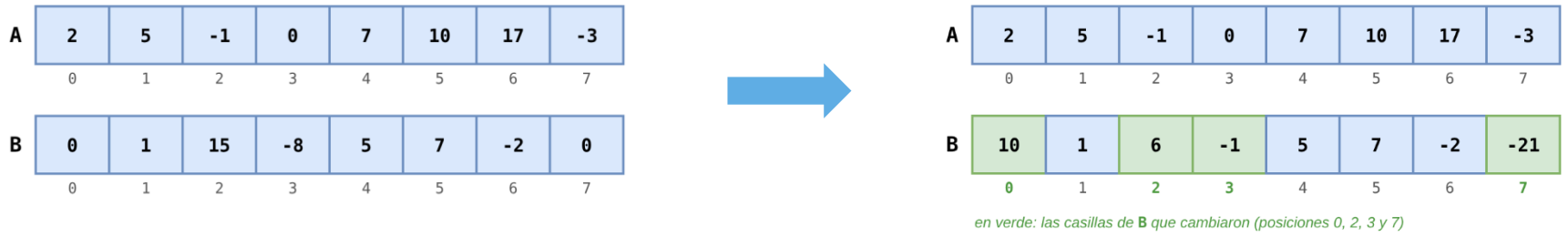
El índice puede ser cualquier expresión entera: **`notas[i]`**, **`notas[i + 1]`**, **`notas[N - 1]`**, e incluso el valor de otro arreglo. En Python se escribe exactamente igual, así que el ejercicio se puede copiar tal cual y comprobar la respuesta.

Ejercicio: ¿Cómo queda el arreglo **B** después de las siguientes instrucciones?

```
A = [2, 5, -1, 0, 7, 10, 17, -3]
B = [0, 1, 15, -8, 5, 7, -2, 0]

B[2] = A[0] * 3
B[3] = B[3] + A[4]
B[A[3]] = A[5] // B[1]
B[A[6] % A[5]] = A[A[7] + A[5]] + B[2] * -3
```

Link simulación



La clave: el índice es una variable

Antes y después

Con índices fijos	Con el índice como variable

`notas[0] = float(input("Nota: "))`
`notas[1] = float(input("Nota: "))`
`notas[2] = float(input("Nota: "))`
`notas[3] = float(input("Nota: "))`
	`for i in range(N):`
	`  notas[i] = float(input("Nota: "))`

Una línea por casilla. Con 40 estudiantes, 40 líneas.	**Una sola línea** sirve para cualquier N.

El patrón que se repite en todo el bloque:

Pseudocódigo	Python

`Para (i = 0, N - 1, 1) Haga`
`  ... notas[i] ...`
`Fin_Para`
	`for i in range(N):`
	`  ... notas[i] ...`

Lo que se hace con este patrón:

- **Llenar**: `Leer(notas[i])` o `notas[i] = expresión`
- **Mostrar**: `Escribir(notas[i])`
- **Acumular**: `suma = suma + notas[i]`
- **Contar**: `Si notas[i] > x Entonces cont = cont + 1`
- ***Próxima clase***: *buscar el mayor, buscar un valor, intercambiar, ordenar.*

Idea clave: “El verdadero beneficio de utilizar un arreglo está en su capacidad para usar una variable como un subíndice.” (Farrell, cap. 6)

Llenar un arreglo

Caso 1: El valor de cada uno de los elementos del arreglo se pasa por teclado.

Pseudocódigo	**Python (simulación)**

`N = 4`
`notas = Arreglo(N)`
`Para (i = 0, N - 1, 1) Haga`
`  Escribir("Nota del estudiante ", i + 1, ": ")`
`  Leer(notas[i])`
`Fin_Para`
	`N = 4`
	`notas = [0.0] * N`
	
	`for i in range(N):`
	`  notas[i] = float(input(f"Nota del estudiante {i + 1}: "))`

Caso 2: El valor de cada uno de los elementos del arreglo proviene de una expresión.

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`aprobo = [Falso, Falso, Falso, Falso]`
 	
`Para (i = 0, N - 1, 1) Haga`
`  aprobo[i] = notas[i] >= 3.0`
`Fin_Para`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	
	`N = 4`
	`aprobo = [False] * N`
	
	`for i in range(N):`
	`  aprobo[i] = notas[i] >= 3.0`

Idea clave: Llenar es darle un valor a cada casilla con un ciclo, y ese valor puede venir de leer o de calcular.

Recorrer y mostrar

Recorrer un arreglo es visitar sus casillas una por una. Mostrar su contenido es el recorrido más simple.

- **Caso 1 – Por índice**: Se usa el índice **`i`**.

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`Para (i = 0, N - 1, 1) Haga`
`  Escribir("Posición ", i, ": ", notas[i])`
`Fin_Para`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	`N = 4`
	`for i in range(N):`
	`  print(f"Posicion {i}: {notas[i]}")`

- **Caso 2 – Por elemento**: Se recibe cada elemento sin usar el índice. Existe en Python y en otros lenguajes (Java, C++), pero no en el pseudocódigo.

Python (simulación)	**Salida**

`for nota in notas:`
`  print("Nota:", nota)`
	`Nota: 3.0`
	`Nota: 4.5`
	`Nota: 2.0`
	`Nota: 4.0`

Comparación:

- **Caso 1**: se conoce la posición **`i`**. Sirve para leer, modificar y saber dónde está cada dato.
- **Caso 2**: se recibe cada valor directamente. Es más corto, pero no da la posición y solo sirve para leer los valores.

Recorrer y acumular

Recorrer un arreglo es visitar sus casillas una por una. En este caso, dentro del recorrido se realizó la actualización de un acumulador (**`suma`**).

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`suma = 0`
`Para (i = 0, N - 1, 1) Haga`
`  suma = suma + notas[i]`
`Fin_Para`
`promedio = suma / N`
`Escribir("Promedio: ", promedio)`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	`N = 4`
	`suma = 0`
	`for i in range(N):`
	`  suma = suma + notas[i]`
	`promedio = suma / N`
	`print(f"Promedio: {promedio:.2f}")`

Prueba de escritorio:

`i`	`notas[i]`	`suma`
—	—	0
0	3.0	3.0
1	4.5	7.5
2	2.0	9.5
3	4.0	13.5

Salida:

```
Promedio: 3.38
```

Idea clave: Como las notas quedan en el arreglo, después de calcular el promedio se pueden volver a recorrer. Eso era lo que faltaba para resolver el problema de la clase 10 (determinar cuántas notas son mayores que el promedio).

Cuidado con salirse del arreglo

Antes y después: el mismo programa, mejor escrito.

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`Para (i = 0, N, 1) Haga`
`  Escribir(notas[i])`
`Fin_Para`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	`N = 4`
	`for i in range(N + 1):`
	`  print(notas[i])`

Salida en pantalla:

```
3.0
4.5
2.0
4.0
IndexError: list index out of range
```

- Las posiciones válidas son **`0`**, **`1`**, **`2`**, **`3`**. En la última vuelta **`i`** vale **`4`**, y **`notas[4]`** no existe.
- Las primeras cuatro notas se muestran bien y el error aparece al final.

Importante: Un recorrido completo va de **`0`** a **`N - 1`**. El error típico es ir hasta **`N`**: **`range(N + 1)`** o **`Para (i = 0, N, 1)`**

Otros recorridos

Recorrido en orden inverso:

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`Para (i = N - 1, 0, -1) Haga`
`  Escribir("Posición ", i, ": ", notas[i])`
`Fin_Para`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	`N = 4`
	`for i in range(N - 1, -1, -1):`
	`  print(f"Posicion {i}: {notas[i]}")`

Salida en pantalla:

```
Posicion 3: 4.0
Posicion 2: 2.0
Posicion 1: 4.5
Posicion 0: 3.0
```

Cuidado: en Python el límite es **`-1`** y no **`0`**, porque range no incluye el final. Con **`range(N - 1, 0, -1)`** el ciclo da **3**, **2**, **1** y se salta la posición **`0`** sin dar error.

Otros recorridos

Variantes del recorrido (solo cambian los valores de `i`):

Recorrido	Pseudocódigo	Python	Posiciones (`N = 6`)
Completo	`Para (i = 0, N - 1, 1)`	`range(N)`	`0, 1, 2, 3, 4, 5`
Inverso	`Para (i = N - 1, 0, -1)`	`range(N - 1, -1, -1)`	`5, 4, 3, 2, 1, 0`
Desde la segunda	`Para (i = 1, N - 1, 1)`	`range(1, N)`	`1, 2, 3, 4, 5`
Hasta la penúltima	`Para (i = 0, N - 2, 1)`	`range(N - 1)`	`0, 1, 2, 3, 4`
Posiciones pares	`Para (i = 0, N - 1, 2)`	`range(0, N, 2)`	`0, 2, 4`
Posiciones impares	`Para (i = 1, N - 1, 2)`	`range(1, N, 2)`	`1, 3, 5`
Primera mitad	`Para (i = 0, N // 2 - 1, 1)`	`range(N // 2)`	`0, 1, 2`
Segunda mitad	`Para (i = N // 2, N - 1, 1)`	`range(N // 2, N)`	`3, 4, 5`

Hasta la penúltima sirve para comparar cada casilla con la siguiente (`notas[i]` con `notas[i + 1]`): así `i + 1` nunca se sale del arreglo.

Idea clave: Para cualquier recorrido hay que responder tres preguntas: desde dónde, hasta dónde y de a cuánto. Eso define **`inicio`**, **`fin`** y **`paso`**.

Comparar vecinos

- En la tabla anterior aparece el recorrido **“Hasta la penúltima”**: **`range(N - 1)`**. A continuación se muestra un caso de uso.
- **Problema**: Un estudiante tuvo estas notas en sus cuatro evaluaciones, en orden: **`notas = [3.0, 4.5, 2.0, 4.0]`**. ¿Su nota subió de una evaluación a la siguiente?

Pseudocódigo	**Python (simulación)**

`notas = [3.0, 4.5, 2.0, 4.0]`
`N = 4`
`Para (i = 0, N - 2, 1) Haga`
`  Si notas[i + 1] > notas[i] Entonces`
`  Escribir("Sube")`
`  Sino`
`  Escribir("Baja o se mantiene")`
`  Fin_Si`
`Fin_Para`
	`notas = [3.0, 4.5, 2.0, 4.0]`
	`N = 4`
	`for i in range(N - 1):`
	`  if notas[i + 1] > notas[i]:`
	`  print("Sube")`
	`  else:`
	`  print("Baja o se mantiene")`

- **¿Por qué el ciclo llega solo hasta N - 2?** Con 4 notas hay **3 comparaciones**. Si **`i`** llegara a **3**, se compararía **`notas[3]`** con **`notas[4]`**, que no existe → **`IndexError`**.

Idea clave: Cuando el cuerpo usa **`notas[i + 1]`**, el recorrido va hasta la penúltima posición.

Asignar una lista no la copia

- **Con listas, = no copia**: **`copia = notas`** le pone otro nombre (un **alias**) a la misma lista. Lo que se cambie con un nombre se ve con el otro.

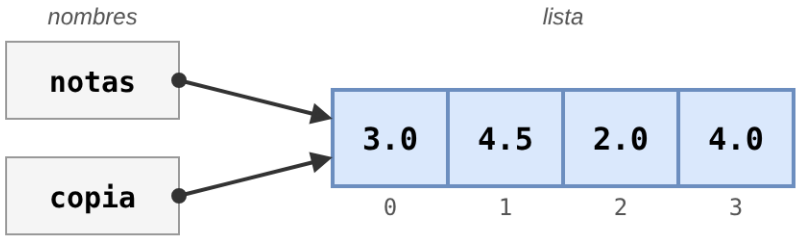
	Con variables (simulación)	**Con listas** (simulación)

Instrucciones	`a = 3.0`
	`b = a`
	`b = 5.0`
		`notas = [3.0, 4.5, 2.0, 4.0]`
		`copia = notas`
		`copia[0] = 5.0`

Resultado	`a = 3.0, b = 5.0`
	
	a no cambió
		`notas = [5.0, 4.5, 2.0, 4.0]`
		`copia = [5.0, 4.5, 2.0, 4.0]`
		
		¡notas también cambió!

1. Antes: se crea el alias

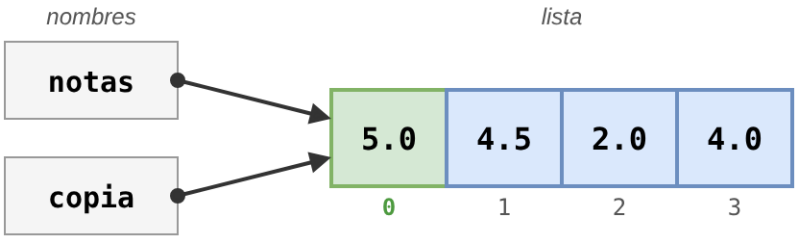
```
copia = notas
```



*dos nombres, **una sola** lista*

2. Después: se cambia una casilla

```
copia[0] = 5.0
```



*se cambió por **`copia`**, pero **`notas[0]`** también vale 5.0*

Clonar una lista con un recorrido

- **Clonar**: crear una lista **nueva** y copiarle los valores **casilla por casilla**. Así cada nombre tiene su propia lista.

	Pseudocódigo	**Python (simulación)**

Instrucciones	`copia = Arreglo(N)`
	`Para (i = 0, N - 1, 1) Haga`
	`  copia[i] = notas[i]`
	`Fin_Para`
	`copia[0] = 5.0`
		`copia = [0.0] * N`
		`for i in range(N):`
		`  copia[i] = notas[i]`
		`copia[0] = 5.0`

Resultado	`notas = [3.0, 4.5, 2.0, 4.0]`
	`copia = [5.0, 4.5, 2.0, 4.0]`
	
	¡notas no cambió!
		`notas = [3.0, 4.5, 2.0, 4.0]`
		`copia = [5.0, 4.5, 2.0, 4.0]`
		
		¡notas no cambió!

1. Antes: se clona la lista

```
copia = [0.0] * N
for i in range(N):
    copia[i] = notas[i]
```



*dos nombres, **dos** listas distintas*

2. Después: se cambia una casilla

```
copia[0] = 5.0
```



solo cambió `copia`: `notas` sigue igual

Clonar una lista con un recorrido

- **Clonar**: crear una lista **nueva** y copiarle los valores **casilla por casilla**. Así cada nombre tiene su propia lista.

	notas[:] (simulación)	**list(notas) (simulación)**
Instrucciones	`copia = notas[:]` `copia[0] = 5.0`	`copia = list(notas)` `copia[0] = 5.0`
Qué hace	Rebanada completa: toma las casillas **desde el inicio hasta el final**	Función que crea una lista **nueva** con los mismos valores
Resultado	`notas = [3.0, 4.5, 2.0, 4.0]` `copia = [5.0, 4.5, 2.0, 4.0]`	`notas = [3.0, 4.5, 2.0, 4.0]` `copia = [5.0, 4.5, 2.0, 4.0]`

1. Antes: se clona la lista

```
copia = notas[:]
# o bien: copia = list(notas)
```



*dos nombres, **dos** listas distintas*

2. Después: se cambia una casilla

```
copia[0] = 5.0
```



*solo cambió **copia**: **notas** sigue igual*

Resumen: arreglos en pseudocódigo y Python

Operación	Pseudocódigo	Python
Crear con N casillas	`A = Arreglo(N)`	`A = [valor_inicial] * N`
Crear con valores	`A = [v0, v1, v2]`	`A = [v0, v1, v2]`
Guardar en una casilla	`A[i] = expresión`	`A[i] = expresión`
Leer desde el teclado	`Leer(A[i])`	`A[i] = float(input())`
Mostrar una casilla	`Escribir(A[i])`	`print(A[i])`

Recorrer completo	`Para (i = 0, N - 1, 1) Haga`
	`  ... A[i] ...`
	`Fin_Para`
		`for i in range(N):`
		`  ... A[i] ...`

Recorrer inverso	`Para (i = N - 1, 0, -1) Haga`	`for i in range(N - 1, -1, -1):`
Recorrer por elemento	—	`for x in A:`
Comparar con el vecino	`Para (i = 0, N - 2, 1) Haga` `  ... A[i] ... A[i + 1] ...`	`for i in range(N - 1):` `  ... A[i] ... A[i + 1] ...`

Clonar	`B = Arreglo(N)`
	`Para (i = 0, N - 1, 1) Haga`
	`  B[i] = A[i]`
	`Fin_Para`
		`B = A[:] o B = list(A)`

Último elemento	`A[N - 1]`	`A[N - 1] o A[-1]`

Observación importante: Toda operación sobre un arreglo completo es un recorrido: un ciclo que hace algo con **`A[i]`**, donde **`i`** va desde un inicio, hasta un fin y de a un paso.

Funciones con listas

Una lista puede **entrar** a una función, **salir** de ella, o **entrar y salir modificada**. A continuación se va a analizar cada caso por separado:

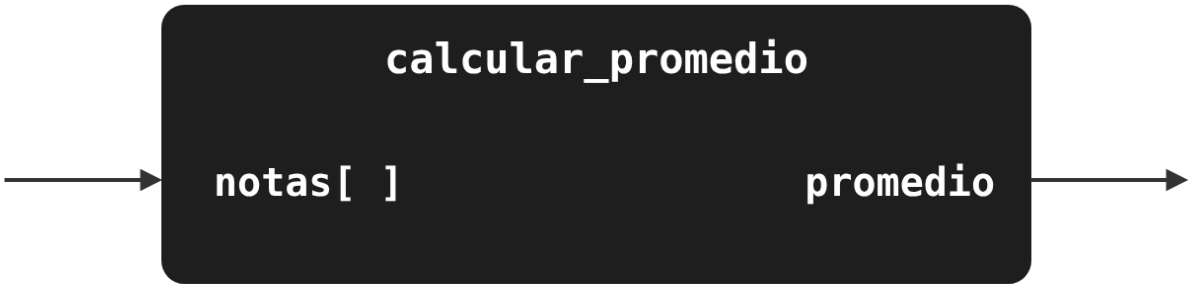


Idea clave: Lo que distingue cada caso es **qué entra, qué sale y si la lista cambia**.

Caso 1: la función recibe una lista y devuelve un valor

- **La función recibe la lista completa como un solo parámetro**: **`notas`**.
- **No conoce `N`**: con **`len(notas)`** sabe cuántos elementos tiene, así sirve para listas de cualquier tamaño.

Definición (simulación):



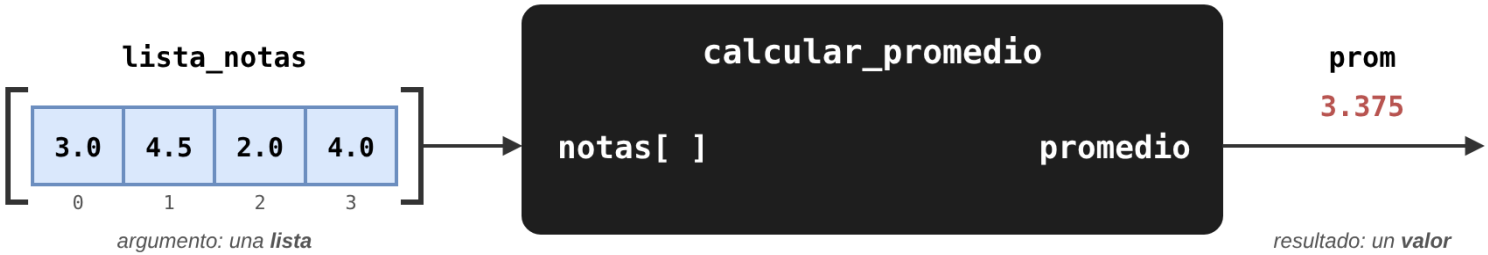
ejemplo: `calcular_promedio([3.0, 4.5, 2.0, 4.0])` *devuelve* `3.375`

```
def calcular_promedio(notas):
    """
    Supone:
        notas es una lista de numeros con al menos un elemento.
    Garantiza:
        Retorna el promedio de las notas (float). No modifica notas.
    """
    suma = 0
    for i in range(len(notas)):
        suma = suma + notas[i]
    promedio = suma / len(notas)
    return promedio
```

El **Garantiza** dice *"No modifica notas"*: la función solo lee la lista.

Caso 1: invocar la función con una lista

- **Al invocar se escribe solo el nombre de la lista**: **`calcular_promedio(lista_notas)`**.
- **Adentro, el parámetro notas es otro nombre para la misma lista**: un **alias**. Como esta función solo lee, no hay efecto secundario.




```
prom = calcular_promedio(lista_notas)
```

Programa (simulación) y **Salida**:

Código invocación

```
lista_notas = [3.0, 4.5, 2.0, 4.0]
prom = calcular_promedio(lista_notas)
print(f"Promedio: {prom:.2f}")
```

Salida en pantalla

```
Promedio: 3.38
```

Observación importante: La misma función sirve para cualquier lista: **`calcular_promedio([4.0, 3.5])`** devuelve **`3.75`**.

Caso 1: probar la función

Pruebas (simulación):

```
assert round(calcular_promedio([3.0, 4.5, 2.0, 4.0]), 3) == 3.375   # tipico
assert calcular_promedio([3.5]) == 3.5                              # limite
assert calcular_promedio([4.0, 4.0, 4.0]) == 4.0                    # especial

datos = [3.0, 4.5, 2.0, 4.0]
calcular_promedio(datos)
assert datos == [3.0, 4.5, 2.0, 4.0]                                # no modifica

print("Todas las pruebas pasaron")
```

Tabla de casos de prueba:

Caso	Entrada	Resultado esperado	Por qué
Típico	`[3.0, 4.5, 2.0, 4.0]`	`3.375`	El ejemplo de la clase
Límite	`[3.5]`	`3.5`	La lista más corta que permite el **Supone**
Especial	`[4.0, 4.0, 4.0]`	`4.0`	Si todas son iguales, el promedio es ese valor
Garantiza	datos antes y después	sin cambios	Verifica *“No modifica notas”*

Idea clave: Probar una función también es comprobar su contrato: lo que **devuelve** y lo que **no cambia**.

Caso 2: la función devuelve una lista

- La función **crea** una lista nueva, la llena y la **devuelve** con **`return`**.
- Recibe un número (**n**, cuántas notas leer) y entrega una **lista**: es el inverso del Caso 1.

Definición:



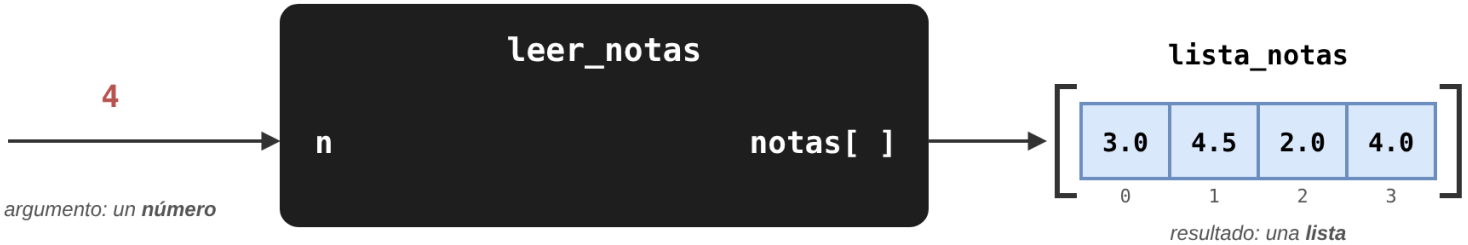
ejemplo: `leer_notas(4)` *lee 4 notas del teclado y devuelve* `[3.0, 4.5, 2.0, 4.0]`

```
def leer_notas(n):
    """
    Supone:
        n es un entero mayor que 0.
    Garantiza:
        Retorna una lista nueva con las n notas leidas, en el orden
        en que se escribieron.
    """
    notas = [0.0] * n
    for i in range(n):
        notas[i] = float(input(f"Nota del estudiante {i + 1}: "))
    return notas
```

Aquí sí se usa n en el range: la función recibe el tamaño porque la lista todavía no existe.

Caso 2: invocar la función

- El resultado se guarda en una variable: **`lista_notas = leer_notas(4)`**.
- La lista se creó **dentro** de la función, pero no se pierde: el `return` la entrega afuera.




```
lista_notas = leer_notas(4)
```

Programa (simulación) y **Salida**:

Código invocación

```
lista_notas = leer_notas(4)
print(lista_notas)
```

Salida en pantalla

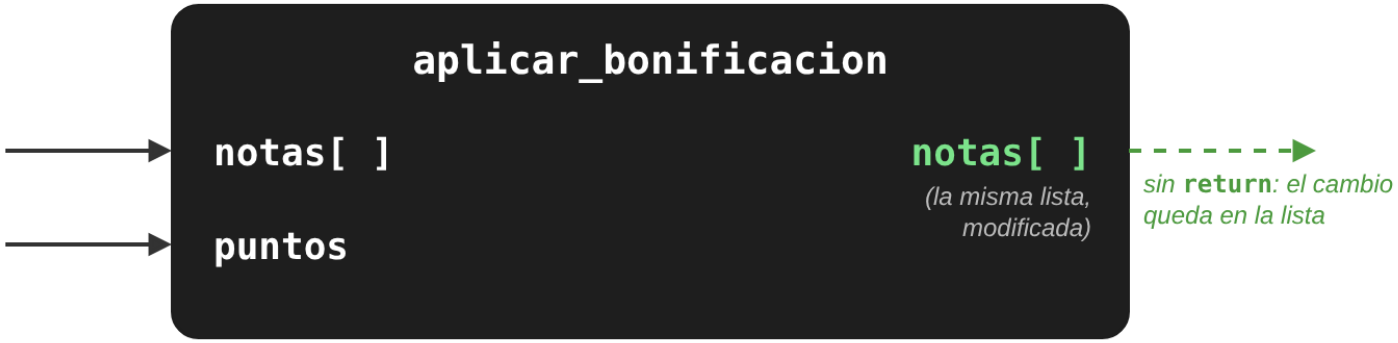
```
Nota del estudiante 1: 3.0
Nota del estudiante 2: 4.5
Nota del estudiante 3: 2.0
Nota del estudiante 4: 4.0
[3.0, 4.5, 2.0, 4.0]
```

Observación: **`leer_notas`** usa **`input()`**, así que no se prueba con **`assert`.** Recuerde que las funciones de entrada y salida se prueban ejecutando el programa.

Caso 3: la función modifica la lista que recibe

- La función **cambia las casillas** de la lista que recibe y **no devuelve nada**.
- Su **Garantiza** debe decirlo claramente: **MODIFICA** la lista.

Definición:



ejemplo: `aplicar_bonificacion(notas, 0.5)` *le suma 0.5 a cada nota de* `notas`

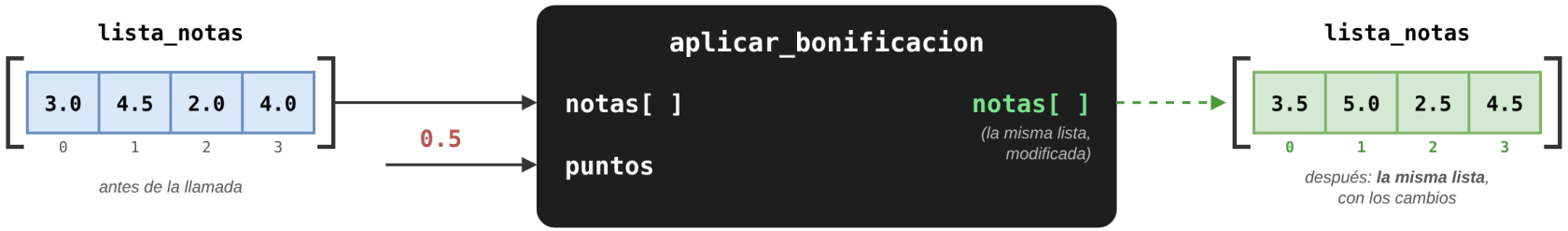
```
NOTA_MAXIMA = 5.0

def aplicar_bonificacion(notas, puntos):
    """
    Supone:
        notas es una lista de numeros entre 0.0 y 5.0, y puntos >= 0.
    Garantiza:
        MODIFICA la lista notas: a cada nota le suma puntos, y si el
        resultado pasa de 5.0, la deja en 5.0. No retorna nada.
    """
    for i in range(len(notas)):
        notas[i] = notas[i] + puntos
        if notas[i] > NOTA_MAXIMA:
            notas[i] = NOTA_MAXIMA
```

Compare el **Garantiza** con el del Caso 1: allá decía *"No modifica notas"*; aquí dice ***"MODIFICA"***.

Caso 3: invocar la función. ¿Por qué cambió la lista afuera?

- La llamada no se asigna a ninguna variable, porque no hay **`return`**: **`aplicar_bonificacion(lista_notas, 0.5)`**.
- Adentro, **`notas`** es un alias de **`lista_notas`** (Asignar una lista no la copia). Por eso lo que la función cambia en notas se ve afuera.




```
aplicar_bonificacion(lista_notas, 0.5)
```

Programa (simulación) y **Salida**:

Código invocación

```
lista_notas = [3.0, 4.5, 2.0, 4.0]
print("Antes: ", lista_notas)
aplicar_bonificacion(lista_notas, 0.5)
print("Despues:", lista_notas)
```

Salida en pantalla

```
Antes:   [3.0, 4.5, 2.0, 4.0]
Despues: [3.5, 5.0, 2.5, 4.5]
```

Observación: Si no se quiere modificar la original, se pasa un clon: **`aplicar_bonificacion(lista_notas[:], 0.5)`**.

Caso 3: probar la función

Pruebas (simulación):

```
datos = [3.0, 4.5, 2.0, 4.0]
aplicar_bonificacion(datos, 0.5)
assert datos == [3.5, 5.0, 2.5, 4.5]      # tipico; 4.5 + 0.5 = 5.0 (limite)

datos = [4.8, 1.0]
aplicar_bonificacion(datos, 0.5)
assert datos == [5.0, 1.5]                # especial: 4.8 + 0.5 pasa de 5.0

datos = [3.0, 4.5]
aplicar_bonificacion(datos, 0)
assert datos == [3.0, 4.5]                # sin bonificacion no cambia
```

Tabla de casos de prueba:

Caso	Lista antes	puntos	Lista después	Por qué
Típico	`[3.0, 4.5, 2.0, 4.0]`	`0.5`	`[3.5, 5.0, 2.5, 4.5]`	El ejemplo de la clase. 4.5 + 0.5 llega **justo** a 5.0 (límite)
Especial	`[4.8, 1.0]`	`0.5`	`[5.0, 1.5]`	4.8 + 0.5 = 5.3 **pasa** de 5.0 y se recorta
Sin cambio	`[3.0, 4.5]`	`0`	`[3.0, 4.5]`	Sin bonificación, la lista queda igual

Idea clave: Para probar una función que modifica una lista: llamar, y luego comparar la lista con **==**.

Resumen de los 3 casos

	Caso 1	**Caso 2**	**Caso 3**
Ejemplo	`calcular_promedio(notas)`	`leer_notas(n)`	`aplicar_bonificacion(notas, puntos)`
Entra	una lista	un valor	una lista (y un valor)
Sale (`return`)	un valor	una lista **nueva**	nada
¿Modifica la lista?	No	— (la crea)	**Sí**
Garantiza dice	*"No modifica notas"*	*"Retorna una lista nueva"*	*"MODIFICA la lista"*
Se invoca	`prom = calcular_promedio(lista_notas)`	`lista_notas = leer_notas(4)`	`aplicar_bonificacion(lista_notas, 0.5)`
Se prueba	**`assert`** sobre el valor que devuelve	Ejecutando el programa (usa **`input`**)	Llamar y **después revisa**

Idea clave: Si una función modifica la lista que recibe, su **Garantiza** debe decirlo, y para no perder la original se le pasa un **clon**.

Volviendo al problema (punto de partida)

Enunciado (el que quedó pendiente en la clase 10):

> Leer las notas de los estudiantes de un curso y mostrar el promedio, cuántas notas son mayores que el promedio y en qué posiciones están.

Entender (Polya):

Entradas	**n** (cantidad de estudiantes) y las n notas
Salidas	El promedio, cuántas notas son mayores que él y sus posiciones
Ejemplo	Notas **`3.0, 4.5, 2.0, 4.0`** → promedio **`3.38`**, **2** notas mayores, en las posiciones `[1, 3]`

¿Por qué no se podía en la clase 10?

- El promedio solo se conoce después de leer todas las notas.
- Con una sola variable **`nota`**, al terminar de leer solo quedaba la última.

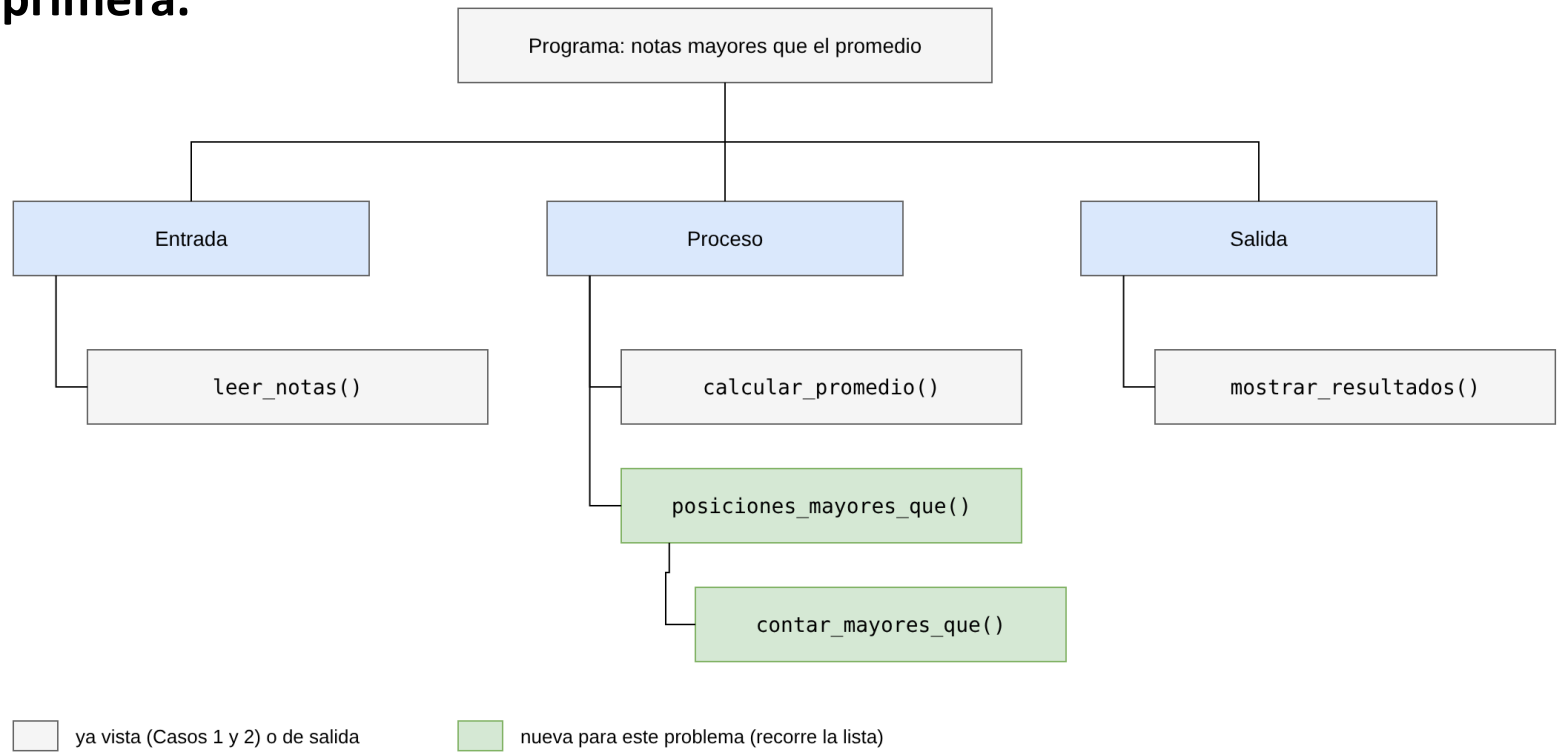
¿Qué cambió?

- Con un arreglo, las notas **quedan guardadas**: primero se calcula el promedio y después **se recorren otra vez** para compararlas.

Idea clave: Un arreglo permite **recorrer los datos más de una vez**. Eso es lo que este problema necesita.

Diseño top-down: ¿qué funciones necesitamos?

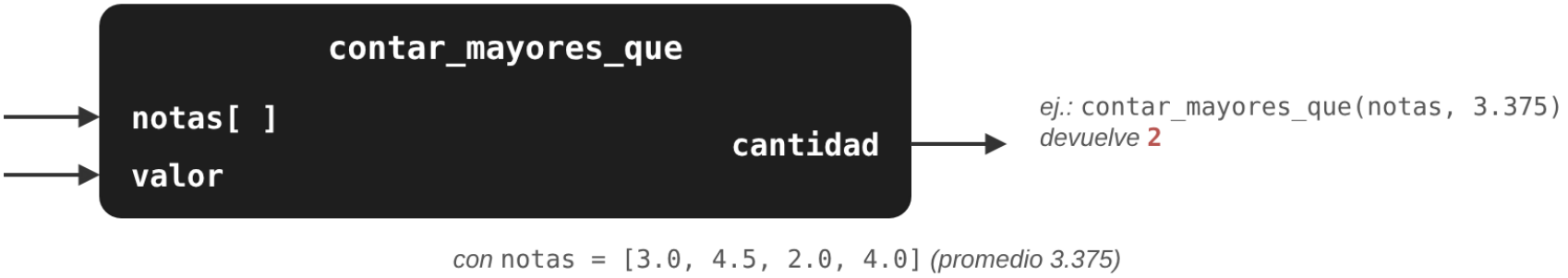
- El problema se divide en **Entrada → Proceso → Salida** (igual que en la clase 10).
- **Dos funciones ya existen**: **`leer_notas`** (Caso 2) y **`calcular_promedio`** (Caso 1). Solo se reutilizan.
- **Faltan dos**: **`contar_mayores_que`** y **`posiciones_mayores_que`**. La segunda usa a **la primera.**



Idea clave: Con **top-down**, el problema nuevo se arma en buena parte con funciones que ya están hechas y probadas.

Especificación: `contar_mayores_que`

- Antes de programar se escribe el **contrato**: qué supone y qué garantiza cada función.
- La funcione **recibe la lista y un valor**, y **no la modifica** (Caso 1).

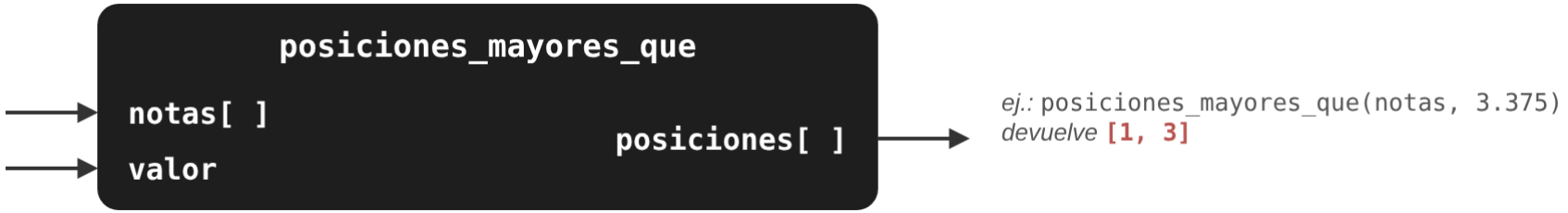



```
def contar_mayores_que(notas, valor):
    """
    Supone:
        notas es una lista de numeros (puede estar vacia) y valor es un numero.
    Garantiza:
        Retorna la cantidad de notas estrictamente mayores que valor (int).
        No modifica notas.
    """
```

Idea clave: Se recibe **`valor`** y no **`promedio`**, así la función sirve para comparar con cualquier número, no solo con el promedio.

Especificación: `posiciones_mayores_que`

- Igual que la anterior, **recibe la lista y un valor** y **no la modifica**.
- Pero devuelve una **lista nueva** con las posiciones, en orden creciente.




```
def posiciones_mayores_que(notas, valor):
    """
    Supone:
        notas es una lista de numeros (puede estar vacia) y valor es un numero.
    Garantiza:
        Retorna una lista nueva con las posiciones (int), en orden creciente,
        de las notas estrictamente mayores que valor. No modifica notas.
    """
```

Idea clave: **`posiciones_mayores_que`** combina dos casos: recibe una lista (**Caso 1**) y devuelve una lista nueva (**Caso 2**).

Implementación: `contar_mayores_que`

- Es el patrón **contar** de *La clave*: el *índice es una variable*, ahora dentro de una función.
- Recorre con **`range(len(notas))`** y cuenta las notas que cumplen la condición.

Código (simulación):

```
def contar_mayores_que(notas, valor):
    cantidad = 0
    for i in range(len(notas)):
        if notas[i] > valor:
            cantidad = cantidad + 1
    return cantidad
```

- **Prueba de escritorio con `notas = [3.0, 4.5, 2.0, 4.0]` y `valor = 3.375`:**

`i`	**`notas[i]`**	**`> 3.375?`**	**`cantidad`**
—	—	—	0
0	3.0	no	0
1	4.5	**sí**	1
2	2.0	no	1
3	4.0	**sí**	2

Implementación: `posiciones_mayores_que`

- Sin **append**, primero hay que saber cuántas posiciones habrá para crear el arreglo del tamaño justo.
- **Se usan dos índices**: **`i`** recorre notas y **`k`** marca la siguiente casilla libre de posiciones.

Código (simulación):

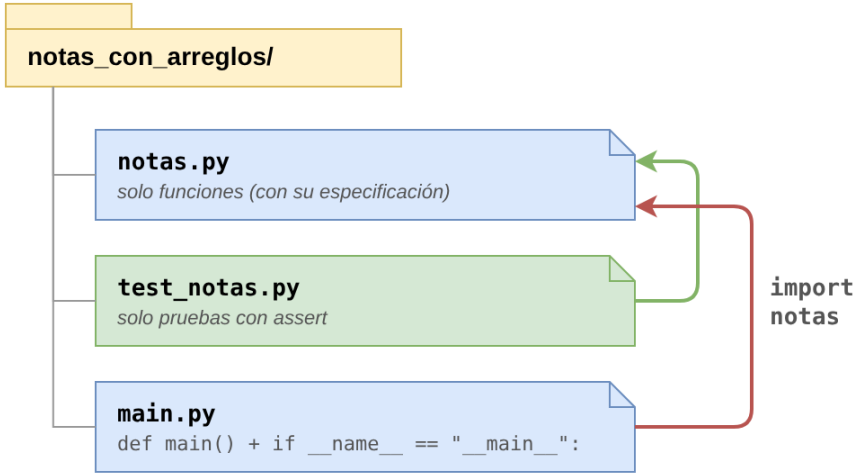
```
def posiciones_mayores_que(notas, valor):
    cantidad = contar_mayores_que(notas, valor)    # 1. contar
    posiciones = [0] * cantidad                    # 2. crear del tamaño exacto
    k = 0                                          # 3. llenar
    for i in range(len(notas)):
        if notas[i] > valor:
            posiciones[k] = i
            k = k + 1
    return posiciones
```

- **Prueba de escritorio con los mismos datos**:

i	**notas[i]**	**> 3.375?**	**k**	**posiciones**
—	—	—	0	[0, 0]
0	3.0	no	0	[0, 0]
1	4.5	**sí**	1	[1, 0]
2	2.0	no	1	[1, 0]
3	4.0	**sí**	2	[1, 3]

El programa completo: tres archivos

- **`notas.py` tiene las cinco funciones**: dos reutilizadas (Casos 1 y 2), dos nuevas y la de salida.
- **`test_notas.py`** y **`main.py`** importan el módulo. Las pruebas y el programa se ejecutan por separado.



*se ejecutan **solo las pruebas***

```
> python test_notas.py
Pruebas de notas: todas pasaron
```

*se ejecuta **solo el programa***

```
> python main.py
Notas mayores que el promedio
Cantidad de estudiantes: 4
Nota del estudiante 1: 3.0
Nota del estudiante 2: 4.5
Nota del estudiante 3: 2.0
Nota del estudiante 4: 4.0
Promedio del curso: 3.38
Notas mayores que el promedio: 2
Posiciones: [1, 3]
```

Idea clave: Es la misma organización del **lab 4**: **funciones**, **pruebas** y **programa principal**, cada uno en su archivo.

El programa principal: **`main.py`**

- El **`while`** garantiza el Supone (**`n > 0`**) antes de llamar a las funciones.
- Las cuatro últimas líneas de **`main()`** son las hojas del **árbol top-down**, en el mismo orden.

```
import notas

def main():
    print("Notas mayores que el promedio")
    n = int(input("Cantidad de estudiantes: "))
    while n <= 0:
        print("ERROR: la cantidad debe ser mayor que 0")
        n = int(input("Cantidad de estudiantes: "))

    lista_notas = notas.leer_notas(n)
    promedio = notas.calcular_promedio(lista_notas)
    posiciones = notas.posiciones_mayores_que(lista_notas, promedio)
    notas.mostrar_resultados(promedio, posiciones)

if __name__ == "__main__":
    main()
```

- **Salida**:

```
Promedio del curso: 3.38
Notas mayores que el promedio: 2
Posiciones: [1, 3]
```

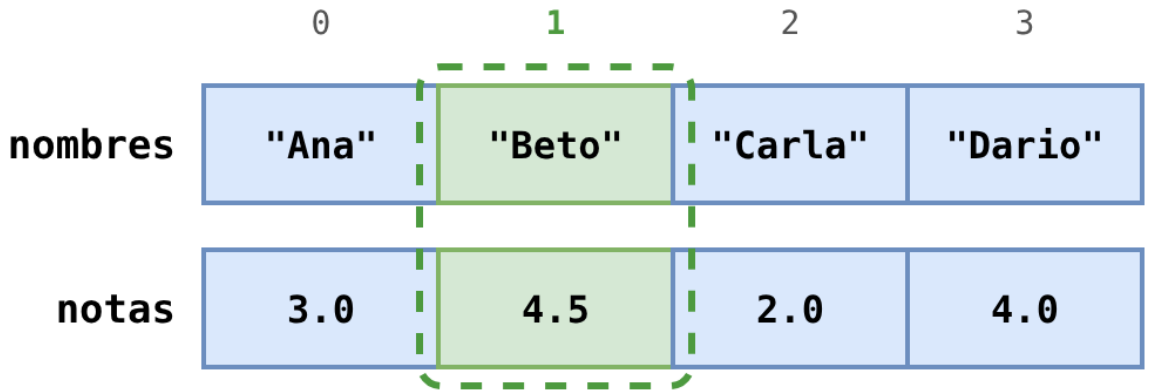
Idea clave: **`main()`** se lee como el árbol: **leer → calcular → buscar → mostrar**. El problema de la clase 10 quedó resuelto.

Resumen de la clase

1. Cuando los datos se usan **después** de leerlos, hay que guardarlos todos: para eso sirven los **arreglos**.
2. Un arreglo guarda muchos valores **del mismo tipo** bajo **un solo nombre**, con **tamaño fijo**. A cada elemento se llega por su **índice**, de `0` a `N - 1`.
3. En Python el arreglo se escribe con una **lista usada con reglas**: `[0.0] * N`, un solo tipo y acceso por posición.
4. La clave es usar el **índice como variable** dentro de un ciclo: llenar, mostrar, acumular, contar y comparar son **recorridos**.
5. Con listas, **=** crea un **alias**, no una copia. Para copiar se **clona**: con un **recorrido**, con `[:]` o con `list()`.
6. Una función puede **recibir** una lista, **devolver** una lista o **modificar** la que recibe. Si la modifica, su **Garantiza** debe decirlo.

Un problema que todavía no se puede resolver bien

Enunciado: Además de las notas, se tienen los **nombres** de los estudiantes. Mostrar **quién** sacó la mejor nota y el **listado del curso ordenado** de la nota mayor a la menor.



*la **misma posición** en los dos arreglos*
*habla del **mismo estudiante**: `Beto` sacó `4.5`*

Si se ordena `notas`, ¿cómo se mantiene cada nombre unido a su nota?

Preguntas:

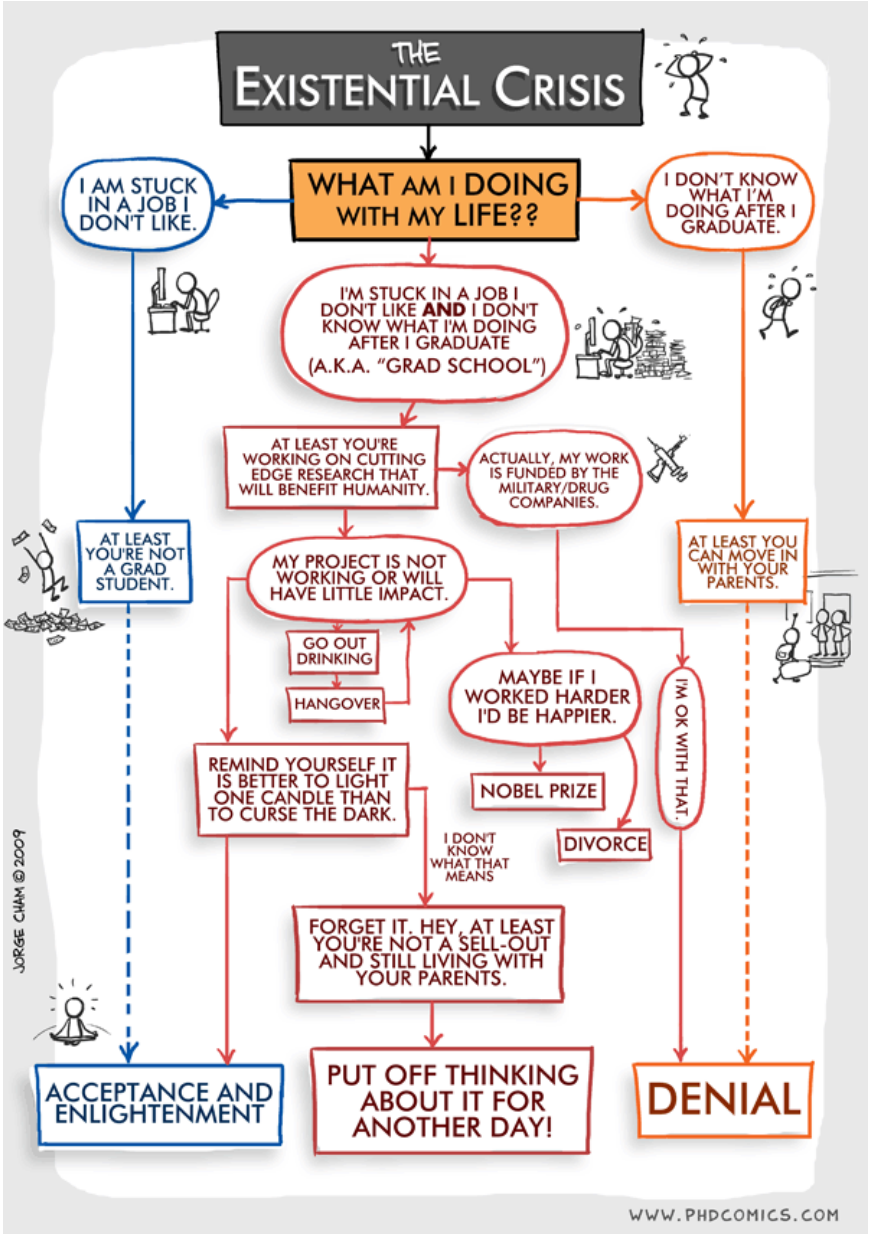
- ¿Cómo se sabe a quién pertenece cada nota?
- ¿Cómo se encuentra la **mayor** nota y **dónde** está?
- Si se **ordenan** las notas, ¿cómo se mantiene cada nombre unido a su nota?

Referencias

- Mera Banguero, C. A. *Guía teórica 05: Arreglos unidimensionales*. Lógica y Representación I, UdeA.
- Harvard **CS50x**, *Lecture 2: Arrays*. https://cs50.harvard.edu/x/notes/2/
- Harvard **CS50P**, *Lecture 2: Loops* (listas). https://cs50.harvard.edu/python/notes/2/
- MIT **6.100L**, *Lecture 11: Aliasing and Cloning*. https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/mit6_100l_f22_lec11.pdf
- Farrell, J. *Introducción a la programación: lógica y diseño*, cap. 6: Arreglos
- Chand John, *How an Array of Integers Gets Stored in Memory*, CS Comics, 2018. https://www.cs-comics.com/how-an-array-of-integers-gets-stored-in-memory/
- Python Tutor. https://pythontutor.com

Eso es todo amigos





https://phdcomics.com/comics/archive.php?comicid=1431